

Adding Two n-Bit Integers

Name: Soumyadip Mahesh

Roll No.: CrS2517

Subject: Cryptography and Security Implementation

July 23, 2026

1 Introduction

The addition of two n -bit binary numbers is one of the fundamental arithmetic operations in digital systems. Suppose A and B are two n -bit integers represented mathematically as $A = (A_{n-1}A_{n-2} \dots A_0)$ and $B = (B_{n-1}B_{n-2} \dots B_0)$, where $A_i, B_i \in \{0, 1\}$. To perform this overall operation, n full adders must be connected in cascade or series to form a complete circuit, commonly known as a Ripple Carry Adder.

2 The Full Adder Building Block

Before analyzing the complete n -bit architecture, it is necessary to establish the operation of a single full adder. A full adder is a combinational logic circuit that computes the arithmetic sum of three binary bits: two operand bits (referred to as A and B , or x and y) and a carry input (C_{in}) propagating from a previous, lower significant stage.

The circuit produces two distinct output bits: a sum bit (S) and a carry-out bit (C_{out}). Two outputs are strictly required because the arithmetic sum of three binary digits can range from 0 to 3, and the binary representation of the decimal values 2 and 3 inherently requires two bits.

Truth Table for a Full Adder

Input x (A)	Input y (B)	Input C_{in}	Output Sum (S)	Output Carry (C_{out})
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Using Karnaugh maps derived from this truth table, the final Boolean logic expressions for the outputs are established:

- **Sum Expression:** $S = x \oplus y \oplus C_{in}$
- **Carry Expression:** $C_{out} = xy + C_{in}(x \oplus y)$ (which can also be written as $AB + AC_{in} + BC_{in}$)

3 Cascading into an n-Bit Adder

To process the full n -bit integers, the full adders are arranged sequentially. For the i -th bit position, where $0 \leq i \leq n-1$, the designated full adder receives the input bits A_i and B_i , alongside the carry-in C_i , to compute the sum S_i and the output carry C_{i+1} .

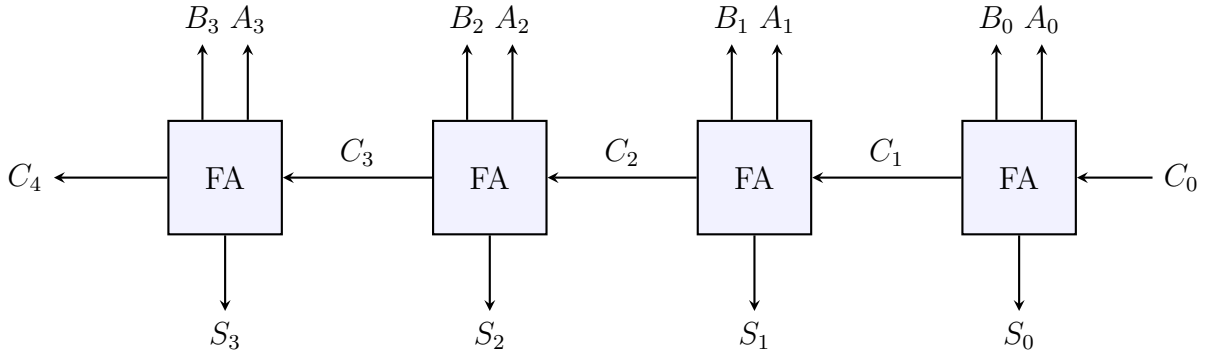


Figure 1: A 4-bit Ripple Carry Adder constructed from sequentially linked Full Adders.

The initial carry input is defined as $C_0 = 0$, strictly because there is no incoming carry generated for the least significant bits. The carry systematically propagates from the least significant stage up to the most significant stage as each full adder computes its result. Once all n full adders have finished processing, the final output sum is formatted as $A + B = (C_n, S_{n-1}, \dots, S_1, S_0)$.

If the final carry output yields $C_n = 1$, it indicates that an overflow condition has occurred. This overflow signifies that the resulting sum mathematically requires more than n bits for accurate representation.

4 Expansion to Subtraction

The standard addition circuit can be efficiently modified into a combined adder-subtractor circuit by placing an XOR gate at each B data input. This is because the subtraction operation $A - B$ can be executed by calculating the 2's complement of B and adding it directly to A .

A mode-of-operation signal (M) dictates the circuit's behavior:

- **Addition** ($M = 0$): The XOR gates output $B \oplus 0 = B$, and the initial input carry is $C_0 = 0$. The full adders receive the correct, unadulterated values of B and perform normal $A + B$ addition.
- **Subtraction** ($M = 1$): The XOR gates output $B \oplus 1 = B'$, generating the 1's complement of the B inputs. The initial carry is set to $C_0 = 1$, effectively adding 1 to the sum and completing the 2's complement conversion, resulting in $A - B$.

5 Algorithmic Complexity

When mapping these digital foundations to high-level representations in languages like C for structural testing or mathematical implementations, tracking computational bounds is critical. The time complexity for the standard Ripple Carry Adder evaluates to $\mathcal{O}(n)$. This linear progression happens because each sequential full adder requires a fixed, continuous amount of time to generate its distinct carry-out. Consequently, stage $i + 1$ is mechanically blocked from computing its final carry until stage i completely finishes its operation.

References

- [1] Self-authored Class Notes, Computing Systems Course, Semester 1.
- [2] M. Morris Mano and Michael D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL*, Pearson, 5th Edition, 2012.